

Contents

Chapter 1: From the Object to the Digital Image	2
1.1 Scene and Optics	2
1.2 Sensor and Colour Filter Array	2
1.3 Readout and Digitisation	2
1.4 Inside a CMOS Image Sensor	3
1.5 The Image Signal Processor	3
1.6 Storage and Display	4
1.7 Further Reading: Sensor Technology	4
Chapter 2: The Pinhole Camera Model	5
2.1 Physical Setup and Coordinate Systems	5
2.2 Perspective Projection	5
2.3 Camera Matrix and Homogeneous Coordinates	5
2.4 Intrinsic Matrix K	5
2.5 Extrinsic Parameters	6
2.6 Full Camera Matrix	6
2.7 Perspective Distortion	6
2.8 Alternative Camera Models	6
2.9 Pose Estimation and Camera Calibration	7
2.10 Camera Matrix Decomposition	7
2.11 Nonlinear Refinement and Lens Distortion	7
2.12 Calibration Methods	7
2.13 Augmented Reality Application	7

Chapter 1: From the Object to the Digital Image

A digital image is the end of a long chain. Light bounces off the scene, a lens focuses it, a sensor turns it into numbers, a processor turns those numbers into an image, and a screen turns the image back into light. The figure shows the whole chain for a digital camera; the sections below go through it step by step.

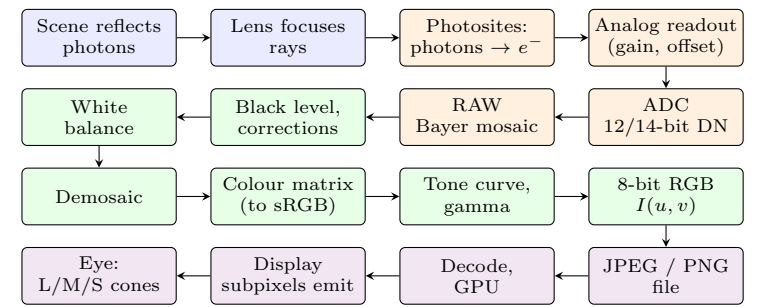


Figure 1: The imaging chain. Blue: optics; orange: sensor and digitisation; green: image signal processor; violet: storage and display.

1.1 Scene and Optics

1. **Light leaves the scene as photons.** A light source (the Sun, a lamp) lights the scene. Objects absorb some of that light and reflect the rest, and some of the reflected photons reach the camera. Different objects reflect different amounts and colours of light:
- a white object reflects most visible light;
 - a black object reflects very little;
 - a red object reflects mostly red light.



Figure 2: An apple.

- The spectrum reaching the camera is the product of the illuminant’s spectrum and the surface’s reflectance - which is why the same object looks different under sunlight and tungsten, and why white balance (step 10) is needed.
2. **The lens focuses the rays.** Every scene point radiates in all directions, so a bare sensor would receive light from every point at every photosite and record only a uniform blur. The lens bends the rays so that photons from one 3-D scene point converge on one sensor location - this is what turns a 3-D scene into a 2-D image. The pinhole model of the next chapter idealises the aperture as a single point and asks only *where* each point lands.

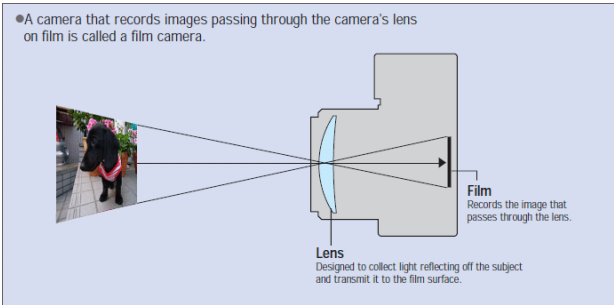


Figure 3: The lens focuses light from the scene onto the sensor.

1.2 Sensor and Colour Filter Array

3. **Photons hit the sensor.** Behind the lens sits a silicon sensor (almost always CMOS today) - a grid of millions of photosensitive *photosites*, from roughly $0.6\mu\text{m}$ pitch in phones to about $6\mu\text{m}$ in full-frame cameras. Each photosite integrates the light arriving at its location during the exposure; this is where the physical world becomes an electrical measurement.

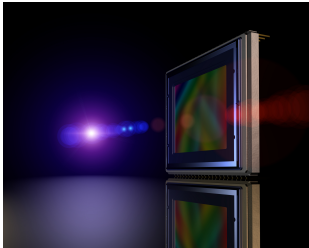


Figure 4: The Sony IMX430 CMOS sensor.

4. **Photons generate electrons.** Via the photoelectric effect, an absorbed photon frees an electron, and the photosite accumulates charge over the exposure:

$$\text{more photons} \rightarrow \text{more electrons} \rightarrow \text{larger signal.}$$

With a quantum efficiency of about 60%, a dark patch delivering 50 photons yields about $30e^-$, while a bright patch delivering 30,000 photons yields about $18,000e^-$. The result is not yet an image - only millions of charge packets, one per photosite. The section on CMOS sensors below looks at how the photosite actually does this.

5. **Colour is sampled by a filter array.** A bare photosite measures *how much* light arrives, not its colour - it cannot report “37% red, 42% green, 21% blue”. A **colour filter array** (CFA) over the sensor makes each photosite respond predominantly to one band. The usual layout is the **Bayer pattern**, with twice as many green sites as red or blue because green carries most of the luminance the eye is sensitive to:

G	R	G	R
B	G	B	G
G	R	G	R
B	G	B	G

Each location therefore measures one colour, and the other two are simply missing. Raw sensor data is a *mosaic*, not an RGB image.

1.3 Readout and Digitisation

6. **The analog signal is read out.** Each photosite’s charge is converted to a voltage and read off the sensor row by row. Along the way the signal is
- amplified by the analog gain (the camera’s ISO setting);
 - shifted by a black-level offset, so that noise below zero is not clipped;
 - corrupted by read noise and other noise sources (see Part 2).

All of these are still analog quantities.

7. **The ADC quantises the signal.** An **analog-to-digital converter** maps each voltage to an integer **digital number** (DN). A 12-bit ADC gives $2^{12} = 4096$ levels, $\text{DN} \in \{0, \dots, 4095\}$; a 14-bit ADC gives 16,384. For example, with a 12-bit ADC,

$$\text{small signal} \rightarrow 217, \quad \text{medium} \rightarrow 1832, \quad \text{large} \rightarrow 3901.$$

From here on the data is digital.

8. **The result is RAW data.** A 2-D array of 12- or 14-bit DNs, one per photosite, still in the Bayer mosaic and in the sensor’s own colour response. The top-left corner of a RAW frame might read

G	R	G	R
1832	924	2103	451
B	G	B	G
388	1790	402	2011

where the letters are *not* stored - they are implied by the known CFA layout. Each position holds a single number. This is what a RAW file stores: sensor *measurements*, not a displayable image.

1.4 Inside a CMOS Image Sensor

Steps 3-7 happen inside the sensor. This section looks at how. The noise sources involved (read noise, kTC noise, dark current, full well and dynamic range, column FPN) are covered in Part 2 and not repeated here.

1.4.1 The Photodiode

Charge generation. Each photosite is a reverse-biased **p-n junction photodiode**. A photon with energy above silicon's band gap ($E_g = 1.12\text{ eV}$) can lift an electron into the conduction band, creating an electron-hole pair. The electric field of the depletion region separates the pair, and the electron is held in the potential well of the photodiode until readout. Silicon therefore responds up to

$$\lambda_{\max} = \frac{hc}{E_g} \approx \frac{1.24\ \mu\text{m} \cdot \text{eV}}{1.12\text{ eV}} \approx 1.1\ \mu\text{m},$$

which is why silicon sensors see near-infrared and why visible cameras carry an **IR-cut filter**.

Absorption depth depends on wavelength. Blue light is absorbed within about $0.5\ \mu\text{m}$ of the surface, red within a few μm , and near-infrared penetrates $10\ \mu\text{m}$ or more. Photodiode depth therefore sets the spectral response, and long-wavelength photons absorbed deep in the substrate can diffuse into a neighbouring pixel (*crosstalk*).

Key figures of merit.

- **Quantum efficiency (QE):** fraction of incident photons that produce a collected electron; about 50-80% at the peak for modern sensors.
- **Full-well capacity:** the most charge a pixel can hold before it saturates. It scales roughly with pixel area, from a few thousand e^- in small phone pixels to 50,000-100,000 e^- in large ones.
- **Fill factor:** fraction of the pixel area that is light-sensitive. Transistors and wiring take up the rest.
- **Conversion gain:** output voltage per electron at the sense node, $\text{CG} = q/C_{\text{FD}}$, typically tens of $\mu\text{V}/e^-$. A smaller sense capacitance means higher gain and lower input-referred noise.

Microlenses. A tiny lens above each pixel focuses light away from the insensitive areas and onto the photodiode, raising the effective fill factor. Toward the edge of the sensor, light arrives at an oblique *chief ray angle*, so the microlenses are shifted slightly toward the centre to compensate. Without that shift, the edges of the image would be darker (a form of vignetting).

1.4.2 Pixel Architectures

What distinguishes CMOS from CCD is that every pixel has its own amplifier - an **active pixel sensor (APS)**.

Passive pixel (PPS). One photodiode and one select transistor. The charge is dumped straight onto a long column line. It is simple, but very noisy, and it is no longer used.

3T active pixel. Three transistors: *reset* (RST) empties the photodiode, a *source follower* (SF) buffers its voltage, and *row select* (RS) connects the pixel to the column line. The photodiode doubles as the sense node, and its reset level is never sampled in the same frame as the signal. The kTC reset noise therefore cannot be cancelled, and dark current at the silicon surface is high.

4T pinned-photodiode pixel. This is the modern standard. A **pinned photodiode** (a buried n-region under a p^+ surface layer) collects the charge. A *transfer gate* (TX) moves the charge to a separate **floating diffusion** (FD), which acts as the sense node. The read sequence is:

- (a) reset the FD and sample the reset level;
- (b) pulse TX to move all the charge onto the FD;
- (c) sample the signal level;
- (d) output the difference.

Both samples share the same reset, so this is true **correlated double sampling** and kTC noise cancels. The pinning layer also shields the photodiode from surface defects, cutting dark current, and the charge transfer is complete, so no residue is left for the next frame (*image lag*).

Shared pixels. Neighbouring photodiodes, typically 2 or 4, each keep their own TX but share one FD, SF, RST and RS. A 4-shared pixel needs 7 transistors for 4 pixels, or 1.75 per pixel. This frees area for the photodiode and is what makes sub- $1\ \mu\text{m}$ pixels possible.

1.4.3 Readout Architecture

A **row decoder** selects one row at a time. Every column has its own amplifier and usually its own ADC, so a whole row is converted in parallel. The column ADC is typically either

- a *single-slope ADC*: a shared ramp voltage is compared against each column while a counter runs, and the count at the crossing is the DN; or
- a *successive-approximation* (SAR) ADC, for higher speed.

Column-parallel conversion is why CMOS reads out fast and at low power. It is also why each column has a slightly different offset, giving the column FPN of Part 2. Because pixels are individually addressable, a CMOS sensor can also read just a window or region of interest, and read it faster.

1.4.4 Rolling versus Global Shutter

Rolling shutter. Rows are reset and read out one after another, so each row's exposure starts a *line time* later than the row above. Reading a full frame takes several to tens of milliseconds. Fast motion therefore produces

- *skew*: vertical lines lean during a pan;
- *wobble* ("jello"): the image shimmers under vibration;
- *partial exposure*: a flash or strobe lights only a band of rows;
- bent propeller blades.

This matters geometrically too: a moving rolling-shutter camera is not a single pinhole camera, because each row has its own pose.

Global shutter. Every pixel starts and stops exposing at the same instant. The charge is then parked in an in-pixel *storage node* until its row is read. The price:

- extra transistors and storage per pixel, so a lower fill factor and full well;
- usually higher noise;
- *parasitic light sensitivity*: stray light leaking into the storage node.

Global shutter is standard in machine vision, robotics and many SLAM cameras, where distortion from motion is unacceptable.

1.5 The Image Signal Processor

The camera (or RAW converter) turns measurements into an image through a fixed pipeline:

RAW $\rightarrow$ black level $\rightarrow$ corrections $\rightarrow$ white balance
 $\rightarrow$ demosaic $\rightarrow$ colour correction $\rightarrow$ tone/gamma $\rightarrow$ RGB.

9. **Black-level and sensor corrections.** Subtract the bias offset so zero light maps to DN 0, then correct defective pixels, fixed-pattern noise and lens shading (vignetting), and apply denoising (see Part 2). These corrections are done while the data is still linear and still a mosaic.
10. **White balance.** Illuminants differ in colour temperature (sunlight, fluorescent, incandescent); per-channel gains compensate so that a physically white surface renders white:

$$R' = aR, \quad G' = bG, \quad B' = cB,$$

usually with $b = 1$. Under tungsten light the raw data is too red, so $a < c$. The gains are estimated automatically, for example with a *grey-world* or *white-patch* assumption, or taken from a preset.

11. **Demosaicing.** Every output pixel needs (R, G, B) , but each photosite measured only one. The missing two components are *estimated* from neighbouring sites. At a red site, for instance, the simplest estimate of green is the mean of its four green neighbours:

$$\hat{G} = \frac{1}{4}(G_{\uparrow} + G_{\downarrow} + G_{\leftarrow} + G_{\rightarrow}).$$

Better methods interpolate *along* edges rather than across them, or exploit the correlation between colour channels. This is why RAW is not simply "three channels of 12-bit RGB", and why demosaicing errors (zippering, false colour, moiré) appear at fine detail.

12. **Colour correction.** The sensor's spectral response differs from any standard colour space, so a 3×3 **colour correction matrix** maps sensor RGB to a standard space such as sRGB:

$$\begin{bmatrix} R' \\ G' \\ B' \end{bmatrix} = M \begin{bmatrix} R \\ G \\ B \end{bmatrix}.$$

M is calibrated per sensor model, typically by photographing a colour chart. Each of its rows sums to 1, so neutral greys stay neutral.

13. **Tone mapping and quantisation to 8 bits.** A tone curve and the gamma (transfer) function compress the linear 12-14-bit data into a display-oriented range, typically $R, G, B \in \{0, \dots, 255\}$. Gamma encoding spends more codes on the dark tones, where the eye is more sensitive; sRGB uses roughly $V = L^{1/2.2}$. The image is now

$$I(u, v) = (R, G, B),$$

with (u, v) the pixel location. For example, $I(100, 200) = (237, 181, 92)$ is an orange-yellow.

1.6 Storage and Display

14. **Compression.** The RGB image is usually encoded, e.g. as JPEG, which is lossy - information is deliberately discarded to shrink the file (see Part 6). The trade-off, for a 24-megapixel camera:
- *RAW*: about 40 MB uncompressed. It keeps the full bit depth and the unprocessed mosaic, so white balance and exposure can be redone later.
 - *JPEG*: about 5-10 MB. It is processed, 8-bit and compressed, so those choices are baked in.
15. **Decoding.** Opening the file, the application decodes it back into an array of pixel values in memory, e.g. $(124, 82, 51)$, $(127, 84, 53)$, $(131, 87, 54), \dots$
16. **The GPU drives the display.** The GPU composites the image into a framebuffer and sends it as an electronic display signal (HDMI, Display-Port). No photons are stored in the file - only numbers.
17. **The display emits light.** Each display pixel has red, green and blue subpixels whose intensities are set by the signal. $(255, 100, 20)$ means bright red, moderate green and dim blue.
- An *LCD* modulates a white backlight through liquid crystals and colour filters.
 - An *OLED* display has subpixels that emit their own light.

The display’s response curve undoes the gamma encoding of step 13, so light output is again roughly linear in scene brightness. The emitted photons reach the eye, whose three cone types (L, M, S) sample the spectrum much as the sensor’s CFA did. This *trichromacy* is why three primaries suffice to reproduce colour.

Geometry versus photometry. The whole chain factors into two questions:

3-D world → optics → 2-D sensor location → pixel value.

Geometry answers “where?” - which pixel (u, v) a 3-D point lands on; this is the camera model of the rest of this part. *Photometry answers “how much light?”* - what DN, and ultimately what RGB value, that pixel records. A digital image needs both.

1.7 Further Reading: Sensor Technology

The topics below are not steps in the pipeline, but round out the picture of how image sensors are built and what alternatives exist.

1.7.1 Sensor Construction: FSI, BSI, Stacked

Front-side illumination (FSI). Light enters through the metal wiring layers above the photodiode. The wiring blocks and scatters light, which gets worse as pixels shrink.

Back-side illumination (BSI; Sony, 2009). The wafer is flipped and thinned, so light reaches the photodiode from the back, with the wiring behind it. This gives higher QE and a wider acceptance angle, and it is essential for small pixels. Crosstalk rises because the silicon is thin. *Deep trench isolation (DTI)* counters it by etching oxide-filled trenches between the pixels.

Stacked sensors (2012 onward). The pixel array sits on one die, and the read-out and logic circuits sit on another die bonded underneath. Direct copper-to-copper *hybrid bonding* can connect them at the pixel or column level. A third DRAM layer can buffer whole frames. Stacking gives the pixel die the whole area, allows much faster readout (shrinking rolling-shutter skew to a few ms) and enables on-sensor processing.

1.7.2 Extending Dynamic Range

- **Dual conversion gain (DCG):** switch extra capacitance onto the FD. High gain gives low read noise in the shadows; low gain holds more charge in the highlights.
- **LOFIC** (lateral overflow integration capacitor): charge that overflows a saturated photodiode is kept in an in-pixel capacitor rather than lost.

- **Split pixels:** a large and a small photodiode side by side, with different sensitivities.
- **Staggered multi-exposure:** long and short exposures are read out in quick succession and merged on the sensor.

1.7.3 CCD versus CMOS

In a **CCD** (charge-coupled device), charge is not converted in the pixel. Instead, clocked gate voltages shift each charge packet from pixel to pixel down the columns, into a serial register, and out to a *single* output amplifier. This bucket-brigade transfer is nearly lossless: the *charge transfer efficiency* is above 0.99999 per step.

	CCD	CMOS (APS)
Charge-to-voltage Readout	one output amplifier serial shift	in every pixel row-select, column-parallel ADC
Speed	slow	fast; windowing possible
Power	high (multi-phase clocks)	low
Integration	separate support chips	ADC, timing, ISP on-chip
Uniformity	excellent (one amplifier)	per-pixel/column FPN
Shutter	global (interline)	rolling, or global at a cost
Artefacts	smear, blooming	rolling-shutter skew

CCDs led on noise and uniformity until the 2000s. Pinned photodiodes, CDS and BSI closed the gap, and CMOS now dominates everywhere except a few scientific niches.

1.7.4 Other Sensor Types

- **sCMOS** (scientific CMOS): read noise around $1 e^-$, high frame rates, and dual-gain readout for wide dynamic range. Used in microscopy and astronomy.
- **EMCCD:** a CCD with a gain register. Impact ionisation multiplies each charge packet *before* the output amplifier, so the effective read noise falls below $1 e^-$ and single photons become detectable. The cost is an excess noise factor of about $\sqrt{2}$.
- **SPAD arrays** (single-photon avalanche diodes): biased above breakdown, so a single photon triggers a detectable avalanche. They count photons digitally with picosecond timing, and are the basis of direct time-of-flight (dToF) LiDAR.
- **Indirect ToF** sensors: the scene is lit with modulated light, and each pixel demodulates the return to measure its phase shift, which gives depth (e.g. Kinect v2).
- **Foveon X3:** three photodiodes stacked at different depths. They separate colour by the wavelength-dependent absorption depth described in the photodiode section, so there is no CFA and no demosaicing.
- **Alternative CFAs:**
 - *Quad Bayer* (Tetracell) groups 2×2 same-colour pixels. They are binned in low light and “remosaiced” to full resolution in good light.
 - *RGBW* and *RYYB* replace some filters with white or yellow ones, letting more light through at some cost in colour accuracy.
- **Event cameras (DVS):** each pixel works asynchronously and emits an event whenever its log intensity changes by more than a threshold. There are no frames. Latency is in microseconds, dynamic range exceeds 120 dB, and motion blur is negligible.
- **Beyond silicon:**
 - *InGaAs* sensors cover the short-wave IR ($0.9\text{-}1.7 \mu\text{m}$), past silicon’s cutoff.
 - *Microbolometers* image thermal long-wave IR ($8\text{-}14 \mu\text{m}$). They are not photon detectors: each pixel is an absorber whose resistance changes as it heats.

Chapter 2: The Pinhole Camera Model

Chapter 1 asked *how much* light a pixel records; this chapter asks *where* a 3-D point lands. The **pinhole camera** answers it with a single point through which all rays pass, and underlies virtually all computer vision geometry.

The complete mapping from a 3-D world point $\mathbf{X}_w$ to a 2-D image point $\mathbf{x}$ is written compactly as

$$\mathbf{x} \sim P \mathbf{X}_w,$$

where P is a 3×4 **camera matrix** and “ $\sim$ ” denotes equality up to a non-zero scale (homogeneous equality).

2.1 Physical Setup and Coordinate Systems

2.1.1 Physical components

- **Aperture (pinhole):** the single point through which all light rays must pass. Making it infinitely small gives a perfectly sharp image but admits no light.
- **Image plane:** the sensor surface at distance f behind the pinhole where the image forms. Each scene point traces a straight ray through the pinhole and lands on the image plane.
- **Camera center (optical center / center of projection):** the pinhole location itself; the geometric origin of the model. All projection rays emanate from here.
- **Focal length f :** the distance from the pinhole to the image plane. A larger f magnifies the image (telephoto); a smaller f widens the field of view (wide-angle).
- **Principal axis (optical axis):** the ray through the camera center perpendicular to the image plane.
- **Principal point (p_x, p_y) :** where the principal axis intersects the image plane. Ideally the image center, but shifts due to manufacturing imperfections.

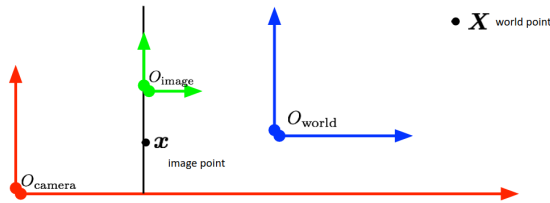
2.1.2 Three coordinate frames

Three reference frames are needed to describe the full projection pipeline:

- **World frame $(\tilde{\mathbf{X}}_w)$:** a fixed global frame in which 3-D scene points are defined. Arbitrary choice.
- **Camera frame $(\tilde{\mathbf{X}}_c)$:** origin at the camera center, z -axis along the principal axis pointing into the scene. The natural frame for projection.
- **Image frame (u, v) :** 2-D pixel coordinates on the sensor, with origin at the image corner (not the principal point).

The full pipeline is:

$$\underbrace{\tilde{\mathbf{X}}_w}_{\text{world}} \xrightarrow{\text{extrinsics } [\mathbf{R}|\mathbf{t}]} \underbrace{\tilde{\mathbf{X}}_c}_{\text{camera}} \xrightarrow{\text{divide by } z, \text{ projection}} \underbrace{(x_n, y_n)}_{\text{normalized}} \xrightarrow{\text{intrinsics } K} \underbrace{(u, v)}_{\text{pixel}}.$$



2.2 Perspective Projection

2.2.1 Derivation from similar triangles

Place the camera center at the origin with the image plane at $z = f$. A 3-D point in camera coordinates $\tilde{\mathbf{X}}_c = (X, Y, Z)^\top$ emits a ray through the origin. That ray intersects the image plane at the 2-D point (x, y) .

By similar triangles along both axes:

$$\frac{x}{f} = \frac{X}{Z} \implies x = \frac{fX}{Z}, \quad \frac{y}{f} = \frac{Y}{Z} \implies y = \frac{fY}{Z}.$$

This is the **perspective projection** (also called *central projection*):

$$\begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \mapsto \begin{pmatrix} fX/Z \\ fY/Z \end{pmatrix}. \quad (1)$$

Two key consequences follow immediately:

- **Depth-dependent scaling:** farther objects appear smaller (magnification $m = f/Z$ decreases with Z).
- **Nonlinearity:** the division by Z makes perspective projection nonlinear in Euclidean coordinates. Homogeneous coordinates linearize it.

2.2.2 Normalized image coordinates ($f = 1$)

Setting $f = 1$ and placing the image plane at $z = 1$ gives the *normalized* (or *canonical*) projection:

$$(X, Y, Z)^\top \mapsto (X/Z, Y/Z)^\top. \quad (2)$$

These **normalized image coordinates** $(x_n, y_n) = (X/Z, Y/Z)$ are independent of any camera-specific parameters and form the intermediate representation before applying the intrinsic matrix K .

2.3 Camera Matrix and Homogeneous Coordinates

2.3.1 Why homogeneous coordinates?

As in Part 4, a 2-D point is $(u, v, 1)^\top$ up to scale; a 3-D point likewise becomes $(X, Y, Z, 1)^\top$. The division by Z is then deferred to a final de-homogenization, making projection *linear*.

2.3.2 Camera as a linear map

A camera maps homogeneous 3-D world coordinates to homogeneous 2-D image coordinates:

$$\mathbf{x} \sim \mathbf{P} \mathbf{X}_w, \quad \mathbf{x} \in \mathbb{P}^2, \quad \mathbf{X}_w \in \mathbb{P}^3, \quad (3)$$

where $\mathbf{P}$ is the 3×4 **camera matrix** (11 DOF after removing the overall scale ambiguity from 12 entries). Expanded:

$$\begin{pmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{pmatrix} = \begin{pmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}, \quad (u, v) = \begin{pmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{pmatrix}.$$

2.3.3 Simplest projection matrix ($f = 1$, camera at origin)

When the camera center coincides with the world origin and $f = 1$:

$$P_0 = [I_{3 \times 3} \mid \mathbf{0}] = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \quad (4)$$

Applying P_0 to $(X, Y, Z, 1)^\top$ yields $(X, Y, Z)^\top$ in homogeneous image coordinates, which de-homogenizes to $(X/Z, Y/Z)^\top$, recovering the normalized projection (2).

2.3.4 General focal length f

Scaling the first two rows by f encodes the image-plane distance:

$$P_f = \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad (X, Y, Z, 1)^\top \mapsto (fX/Z, fY/Z)^\top, \quad (5)$$

recovering the full perspective equation (1).

2.4 Intrinsic Matrix K

The **intrinsic matrix** K converts normalized image coordinates (x_n, y_n) to pixel coordinates (u, v) . It bundles all camera-internal geometric parameters.

2.4.1 Principal-point offset

The image origin is typically at the sensor corner, not at the principal point. Adding offset (p_x, p_y) in pixels:

$$u = f x_n + p_x, \quad v = f y_n + p_y. \quad (6)$$

This gives the standard K for a camera with square pixels:

$$K = \begin{pmatrix} f & 0 & p_x \\ 0 & f & p_y \\ 0 & 0 & 1 \end{pmatrix}. \quad (7)$$

Intrinsic DOF: f (1) + p_x, p_y (2) = 3 intrinsic parameters. With 6 extrinsic DOF ($\mathbf{R}$: 3, $\mathbf{t}$: 3), the standard model has **9 DOF** total.

2.4.2 CCD camera — non-square pixels

Digital sensors may have non-square pixels, giving separate effective focal lengths f_x and f_y (in pixels) along each axis:

$$K = \begin{pmatrix} f_x & 0 & p_x \\ 0 & f_y & p_y \\ 0 & 0 & 1 \end{pmatrix}, \quad f_x = f s_x, \quad f_y = f s_y, \quad (8)$$

where s_x, s_y are the pixel densities (pixels per meter). Adding f_y as a free parameter brings the intrinsic count to 4, giving **10 DOF** total.

2.4.3 Finite projective camera — sensor skew

If the pixel axes are not exactly perpendicular (e.g., due to manufacturing imperfections), a **skew** parameter s couples the x and y directions:

$$K = \begin{pmatrix} f_x & s & p_x \\ 0 & f_y & p_y \\ 0 & 0 & 1 \end{pmatrix}, \quad s \neq 0. \quad (9)$$

In practice $s \approx 0$ for modern sensors, but the general model with all 5 intrinsic parameters free gives **11 DOF** total. This is the most general *finite* camera (infinite cameras such as orthographic are excluded).

2.4.4 Degrees-of-freedom summary

Camera model	Intrinsic params	Extra	Total DOF
Standard pinhole	f, p_x, p_y	—	9
CCD (non-square pixels)	f_x, f_y, p_x, p_y	f_y	10
Finite projective (skew)	f_x, f_y, s, p_x, p_y	s	11

Extrinsic DOF (6) = rotation (3) + translation (3) in all cases.

2.5 Extrinsic Parameters

2.5.1 World-to-camera coordinate transform

The **extrinsic parameters** describe the camera's pose (position and orientation) in the world. They transform world coordinates into camera coordinates before projection.

Let $\tilde{\mathbf{C}} \in \mathbb{R}^3$ be the camera center's position in world coordinates, and $\mathbf{R} \in SO(3)$ be the 3×3 rotation matrix whose columns are the world-frame directions of the camera's x -, y -, and z -axes.

The world-to-camera transform in Euclidean coordinates is:

$$\tilde{\mathbf{X}}_c = \mathbf{R}(\tilde{\mathbf{X}}_w - \tilde{\mathbf{C}}) = \mathbf{R} \tilde{\mathbf{X}}_w \underbrace{- \mathbf{R} \tilde{\mathbf{C}}}_{\mathbf{t}}. \quad (10)$$

Key insight: $\mathbf{t} = -\mathbf{R} \tilde{\mathbf{C}}$ is *not* the camera position but rather the position of the world origin in camera coordinates.

In homogeneous block form (4×4), this is an invertible rigid-body transform:

$$\mathbf{X}_c = \underbrace{\begin{pmatrix} \mathbf{R} & -\mathbf{R} \tilde{\mathbf{C}} \\ \mathbf{0}^\top & 1 \end{pmatrix}}_{[\mathbf{R}|\mathbf{t}]_{\text{homog.}}} \mathbf{X}_w. \quad (11)$$

2.5.2 Recovering the camera center from $\mathbf{t}$

Given the compact parameterization $P = K[\mathbf{R} | \mathbf{t}]$, the camera center in world coordinates is recovered as:

$$\tilde{\mathbf{C}} = -\mathbf{R}^\top \mathbf{t}. \quad (12)$$

2.6 Full Camera Matrix

2.6.1 Assembling the pipeline

The complete projection from world point to pixel is:

$$\mathbf{X}_c = [\mathbf{R} | \mathbf{t}] \mathbf{X}_w \quad (\text{extrinsics: world} \rightarrow \text{camera}) \quad (13)$$

$$\mathbf{x}_n = P_0 \mathbf{X}_c \quad (\text{ideal projection with } f = 1) \quad (14)$$

$$\mathbf{x} = K \mathbf{x}_n \quad (\text{intrinsics: normalized} \rightarrow \text{pixel}). \quad (15)$$

Chaining these three steps gives the canonical decomposition of P :

$$P = K [I | \mathbf{0}] \begin{pmatrix} \mathbf{R} & -\mathbf{R} \tilde{\mathbf{C}} \\ \mathbf{0}^\top & 1 \end{pmatrix} = K [\mathbf{R} | \mathbf{t}], \quad \mathbf{t} = -\mathbf{R} \tilde{\mathbf{C}}. \quad (16)$$

The left 3×3 block of P is $M = K\mathbf{R}$, the product of an upper-triangular matrix (K) and an orthogonal matrix ($\mathbf{R}$). This structure enables later decomposition via QR.

2.6.2 Block-size summary

Block	Size	Structure	Physical meaning
K	3×3	upper-triangular	intrinsic camera geometry
$\mathbf{R}$	3×3	orthogonal $SO(3)$	camera orientation in world
$\mathbf{t}$	3×1	vector	world origin in camera frame
P	3×4	general rank-3	full projection matrix

2.7 Perspective Distortion

2.7.1 Depth-dependent magnification

The image magnification at depth Z is:

$$m(Z) = \frac{f}{Z}. \quad (17)$$

Because m depends on Z , objects at different depths are scaled differently. This is **perspective distortion**: parallel lines converge to *vanishing points*, and depth changes cause apparent size changes even for rigid objects.

2.7.2 The dolly-zoom (Vertigo) effect

Doubling f while simultaneously doubling Z keeps the foreground subject at the same image size ($m = f/Z = \text{const}$), but background objects — at a different depth Z' — are now magnified by $f'/(Z') = 2f/Z'$, making them appear compressed relative to the foreground. This is the basis of the **dolly-zoom** (“Vertigo effect”) in cinematography.

2.7.3 Foreshortening and vanishing points

A set of parallel lines in the world with direction vector $\mathbf{d}$ converges in the image to a single **vanishing point** at $\mathbf{v} = K\mathbf{R}\mathbf{d}$ (in homogeneous coordinates). The horizon line is the set of all vanishing points for horizontal directions, lying at the image row corresponding to the camera's eye level.

2.8 Alternative Camera Models

2.8.1 Weak perspective (scaled orthographic)

When the scene depth variation ΔZ is small compared to the average depth Z_0 (i.e., $\Delta Z \ll Z_0$), all points are approximated as being at the same depth Z_0 . The depth-dependent magnification f/Z is then replaced by a single constant scale factor $m = f/Z_0$:

$$(X, Y, Z)^\top \mapsto (mX, mY)^\top, \quad m = \frac{f}{Z_0}, \quad (18)$$

$$P_{\text{weak}} = \begin{pmatrix} m & 0 & 0 & 0 \\ 0 & m & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (19)$$

This is valid when (1) the scene is very far from the camera, or (2) a **telecentric lens** is used (pinhole placed at the front focal point, admitting only rays parallel to the principal axis).

2.8.2 Orthographic projection

The special case of weak perspective with $m = 1$ and no origin offset gives pure **orthographic projection**: depth is entirely ignored, and world x, y coordinates map directly to image coordinates.

$$P_{\text{ortho}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (20)$$

Orthographic projection preserves parallel lines, distances, and ratios along any direction — properties lost in perspective projection. It is accurate for engineering/CAD drawings and telephoto photography of distant objects.

2.9 Pose Estimation and Camera Calibration

Goal: Given N point correspondences $\{(\mathbf{X}_i, \mathbf{x}_i)\}_{i=1}^N$ between known 3-D world points and observed 2-D image points, estimate the 3×4 camera matrix P .

2.9.1 Projection equations in inhomogeneous coordinates

Let $\mathbf{m}_j^\top$ denote the j -th row of P (a 1×4 vector). The projection $\mathbf{x}_i = P \mathbf{X}_i$ in homogeneous form de-homogenizes to:

$$u_i = \frac{\mathbf{m}_1^\top \mathbf{X}_i}{\mathbf{m}_3^\top \mathbf{X}_i}, \quad v_i = \frac{\mathbf{m}_2^\top \mathbf{X}_i}{\mathbf{m}_3^\top \mathbf{X}_i}. \quad (21)$$

These are rational (nonlinear) functions of P 's entries due to the shared denominator $\mathbf{m}_3^\top \mathbf{X}_i$.

2.9.2 Linearization: Direct Linear Transform (DLT)

Cross-multiplying (21) clears the denominator and yields two *linear* equations per correspondence:

$$\mathbf{m}_1^\top \mathbf{X}_i - u_i \mathbf{m}_3^\top \mathbf{X}_i = 0, \quad (22)$$

$$\mathbf{m}_2^\top \mathbf{X}_i - v_i \mathbf{m}_3^\top \mathbf{X}_i = 0. \quad (23)$$

Stacking these for all N correspondences into a matrix $A \in \mathbb{R}^{2N \times 12}$:

$$A \mathbf{p} = \mathbf{0}, \quad \mathbf{p} = \text{vec}(P) \in \mathbb{R}^{12}. \quad (24)$$

This is a homogeneous linear system: the camera matrix P lies in the null space of A . Since P has 11 DOF (12 entries, 1 scale), each correspondence contributes 2 constraints, so $N \geq 6$ correspondences are needed (actually $N = 6$ gives exactly 11 constraints; more gives an overdetermined system solved in the least-squares sense).

2.9.3 Solution by SVD

The minimum-norm solution (constrained to $\|\mathbf{p}\| = 1$ to avoid the trivial $\mathbf{p} = \mathbf{0}$) is:

$$\hat{\mathbf{p}} = \arg \min_{\|\mathbf{p}\|=1} \|A\mathbf{p}\|^2 = \text{last column of } V \text{ in } A = U\Sigma V^\top. \quad (25)$$

Equivalently, $\hat{\mathbf{p}}$ is the eigenvector of $A^\top A$ corresponding to its *smallest* eigenvalue. This is the **DLT algorithm**.

2.10 Camera Matrix Decomposition

Given an estimated P , it is useful to recover the physical parameters K , $\mathbf{R}$, and $\tilde{\mathbf{C}}$ separately.

2.10.1 Recovering the camera center $\tilde{\mathbf{C}}$

By definition, the camera center projects to the zero vector under P :

$$P \tilde{\mathbf{C}}_h = \mathbf{0} \implies \tilde{\mathbf{C}}_h = \text{null}(P). \quad (26)$$

Numerically, $\tilde{\mathbf{C}}_h$ is the right singular vector of P corresponding to the smallest singular value (from $P = U\Sigma V^\top$, take the last column of V). De-homogenize to get $\tilde{\mathbf{C}}$.

2.10.2 Recovering intrinsics K and rotation $\mathbf{R}$

Write $P = [M \mid \mathbf{p}_4]$ where $M = K\mathbf{R}$ is the left 3×3 block. Since K is upper-triangular with positive diagonal and $\mathbf{R}$ is orthogonal, their product M has a unique factorization via **RQ decomposition** (a variant of QR):

$$M = K\mathbf{R} \xrightarrow{\text{RQ}} K' \text{ (upper-triangular, positive diagonal)}, \quad \mathbf{R} \text{ (orthogonal)}. \quad (27)$$

Finally, $\mathbf{t} = K^{-1}\mathbf{p}_4$, and the camera center is $\tilde{\mathbf{C}} = -\mathbf{R}^\top \mathbf{t}$.

2.11 Nonlinear Refinement and Lens Distortion

2.11.1 Reprojection error

The DLT solution minimizes an algebraic error that has no direct geometric meaning. The preferred objective is the **reprojection error** — the sum of squared Euclidean distances between observed image points and their projections under the estimated P :

$$E(K, \mathbf{R}, \mathbf{t}) = \sum_{i=1}^N \left[\left(u_i - \frac{\mathbf{m}_1^\top \mathbf{X}_i}{\mathbf{m}_3^\top \mathbf{X}_i} \right)^2 + \left(v_i - \frac{\mathbf{m}_2^\top \mathbf{X}_i}{\mathbf{m}_3^\top \mathbf{X}_i} \right)^2 \right]. \quad (28)$$

Minimized over K , $\mathbf{R}$, $\mathbf{t}$ by iterative nonlinear optimization (typically Levenberg–Marquardt), initialized from the DLT solution.

2.11.2 Radial lens distortion

Real lenses deviate from the ideal pinhole model. The dominant imperfection is **radial distortion**: image points are displaced radially from the principal point. Let (x_n, y_n) be ideal normalized coordinates and $r^2 = x_n^2 + y_n^2$. The distorted normalized coordinates are:

$$x_d = x_n (1 + k_1 r^2 + k_2 r^4 + k_3 r^6 + \dots), \quad (29)$$

$$y_d = y_n (1 + k_1 r^2 + k_2 r^4 + k_3 r^6 + \dots), \quad (30)$$

where k_1, k_2, k_3 are **radial distortion coefficients** estimated during calibration.

- **Barrel distortion** ($k_1 < 0$): image bulges outward; straight lines bow away from center. Common in wide-angle lenses.
- **Pincushion distortion** ($k_1 > 0$): image pinches inward; straight lines bow toward center. Common in telephoto lenses.

The distorted reprojection error adds distortion correction to the pipeline: $(x_n, y_n) \rightarrow (x_d, y_d) \rightarrow K \rightarrow (u, v)$.

2.12 Calibration Methods

2.12.1 Reference-object (3-D target) calibration

Place an object of precisely known 3-D geometry in the scene (e.g., a calibration rig with marked corner points), identify corresponding 2-D image points, and solve via DLT.

- **Pros:** Closed-form solution; straightforward to implement.
- **Cons:** Requires a precisely manufactured 3-D object; does not model radial distortion without extension; cannot impose known constraints (e.g., known aspect ratio, zero skew).

2.12.2 Multi-plane (Zhang's) calibration

Photograph a flat planar pattern (e.g., checkerboard) from at least 3 different viewpoints. Corner positions on the plane are known in 2-D pattern coordinates; their 3-D positions are inferred from the known planar structure. Each viewpoint provides a **homography** (a 3×3 projective transform between the plane and the image) from which constraints on K are extracted.

- **Pros:** Only requires a printed checkerboard; positions and orientations of the board need not be known; handles radial distortion; implemented in MATLAB (Bouguet toolbox) and OpenCV.
- **Cons:** Requires at least 3 views; involves nonlinear optimization for final refinement.

2.13 Augmented Reality Application

The full camera pipeline is illustrated concretely by a simple AR system:

1. **Detect** known 2-D feature points on a flat AR marker (e.g., a QR-code border or checkerboard).
2. **Estimate pose** $[\mathbf{R} \mid \mathbf{t}]$ from the 2-D $\leftrightarrow$ 3-D correspondences using DLT followed by Levenberg–Marquardt refinement on reprojection error. K is pre-calibrated.
3. **Render virtual content** by projecting 3-D virtual geometry through the estimated $P = K[\mathbf{R} \mid \mathbf{t}]$:

$$\mathbf{x}_{\text{virtual}} \sim P \mathbf{X}_{3D}.$$

4. **Composite** the rendered virtual image onto the live camera feed, producing the AR output.

The quality of the overlay depends directly on the accuracy of K (calibration) and $[\mathbf{R} \mid \mathbf{t}]$ (pose estimation).